Disk Scheduling Algorithm Visualizer / Simulator

Low-Level Operating System Simulation & Performance Analysis of Disk I/O Scheduling Heuristics

Made with Napkin

The Seek Time Bottleneck

Addressing mechanical latency in secondary storage operations



Mechanical Constraints

Seek time (positioning the R/W head) is the dominant factor in disk I/O latency. Formula: T_seek = m * n + s, where n is cylinders crossed.



System Impact

In multi-programming environments, unoptimized request queues lead to "arm thrashing" and degraded system throughput.



Objective

Quantify efficiency by calculating Total Head Movement (THM). Metric: Average Seek Time = THM / Total Requests.

Made with Napkin

Made with Napkin

Technical Implementation Details

Memory Management

Static allocation with MAX_REQUESTS (e.g., 100) to ensure O(1) memory access and avoid heap fragmentation.

Mathematical Precision

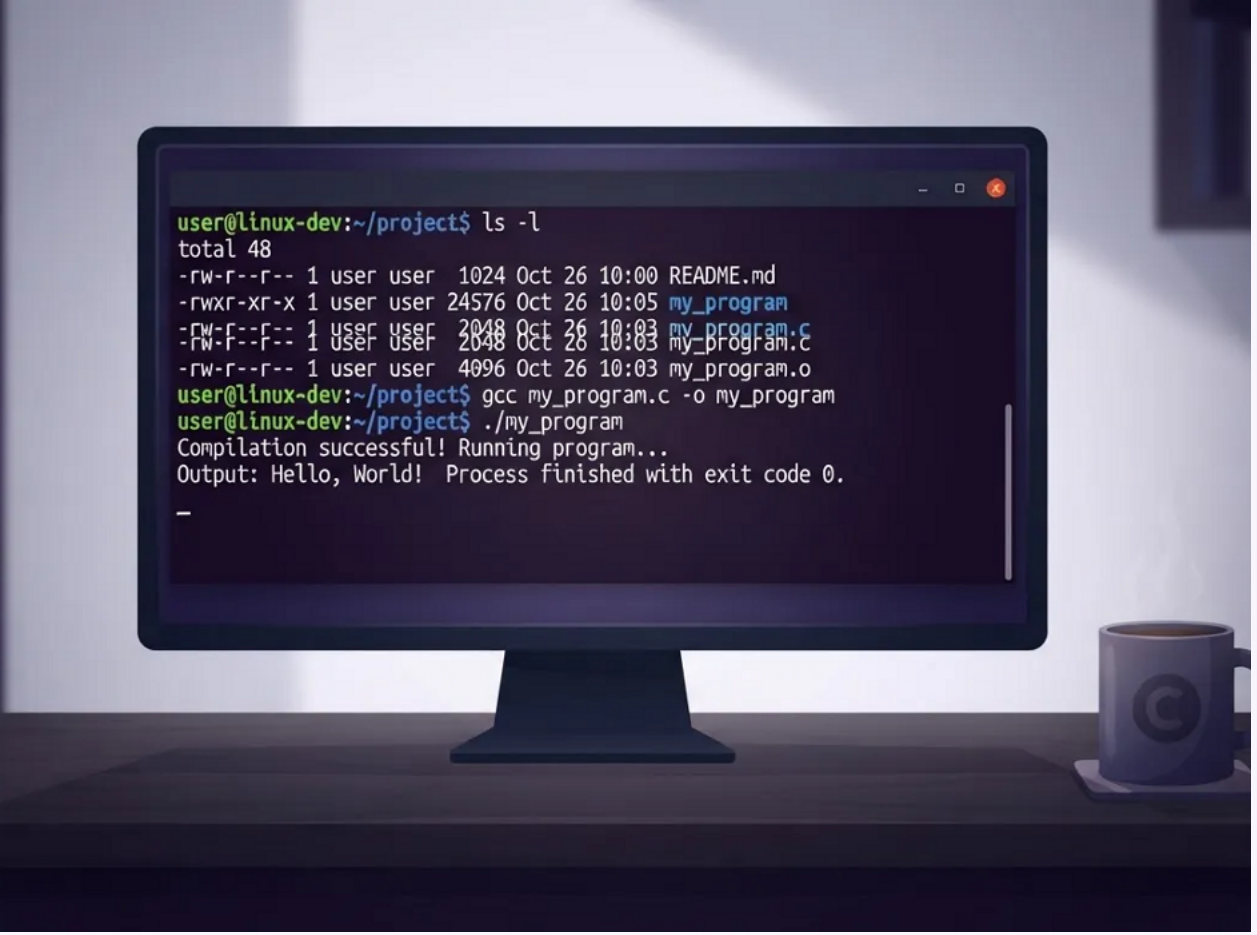
Utilization of abs() from <stdlib.h> for displacement. Logic: diff = abs(queue[i] - current_head).

Input Validation

Robust handling of queue size, head position, and cylinder range to prevent buffer overflows or logical errors.

C99 standards and POSIX-compliant development

Made with Napkin



Development Stack

Tools and environment for systems-level programming

Environment

Ubuntu 22.04 LTS environment utilizing POSIX system calls for I/O simulation.

Toolchain

GCC 11.4.0 with -Wall -Wextra flags for strict error checking and -lm for math.h.

Workflow

Git-based version control with a structured repository for reproducibility and documentation.

Made with Napkin

Execution & Test Methodology

Verifying algorithmic correctness through terminal execution



01

Compilation

gcc disc_scheduling.c -o disk_scheduler -lm

02

Test Parameters

Initial Head: 50 | Queue: {45, 82, 15} | Range: [0, 199] cylinders.

03

FCFS Path

50→45→82→15. THM = |45-50| + |82-45| + |15-82| = 5+37+67 = 109.

04

SSTF Path

50→45→15→82. THM = |45-50| + |15-45| + |82-15| = 5+30+67 = 102.

Made with Napkin

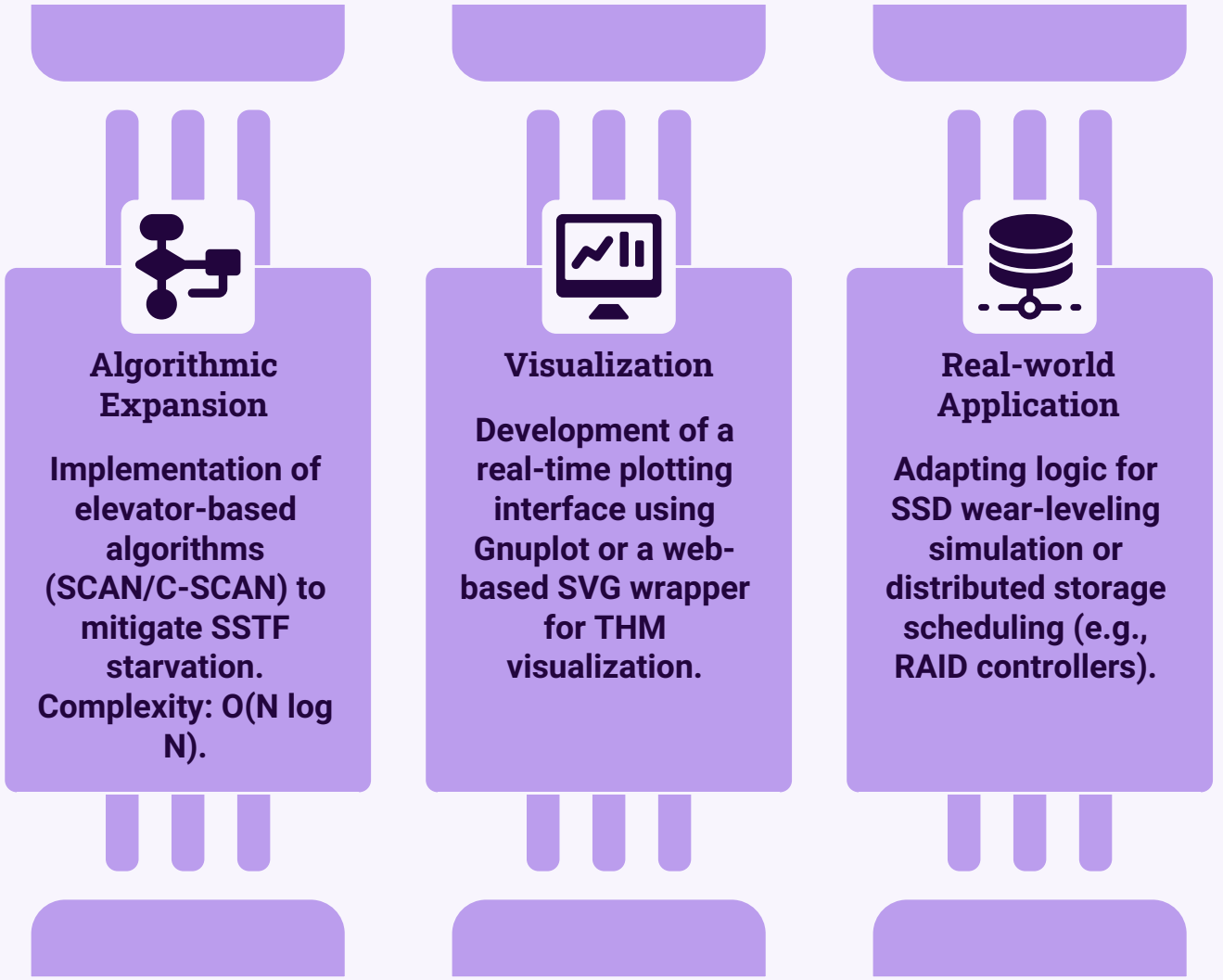
Performance Analysis & Trade-offs

Conclusion: SSTF is superior for throughput, while FCFS is preferred for predictable response times.

Algorithm	FCFS	SSTF
Analysis	Absolute fairness (no starvation). Performance: High variance in seek times; THM is typically maximal. O(N) complexity.	Reduces seek time (e.g., 28.5% reduction). Risk: Starvation of requests at cylinder extremes. O(N^2) complexity.

Made with Napkin

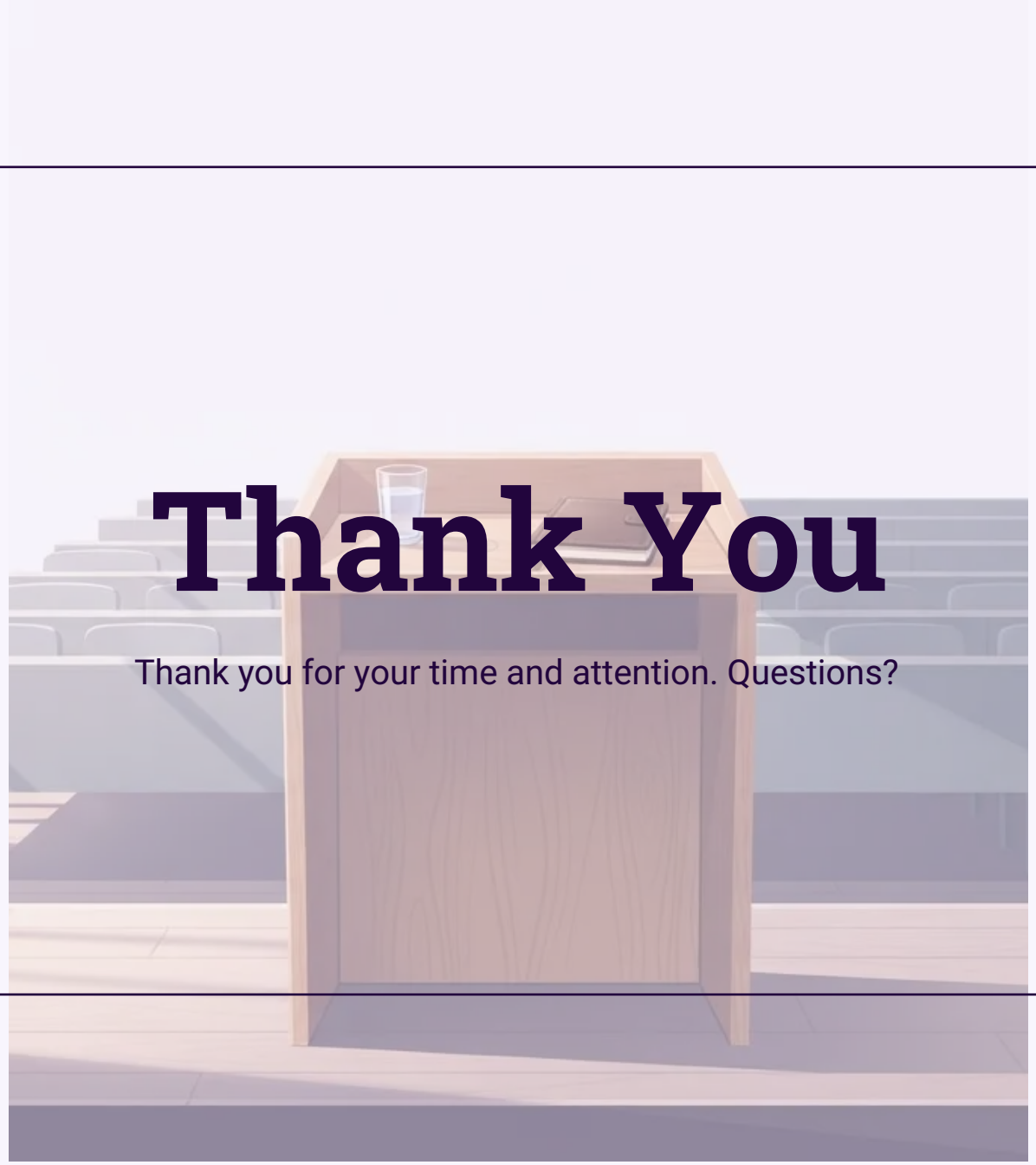
Future Scope & Enhancements



Made with Napkin

Thank You

Thank you for your time and attention. Questions?



Made with Napkin